

Diffusion

Denoising Diffusion Probabilistic Models

Jonathan Ho, Ajay Jain, Pieter Abbeel, NeurIPS 2020

1. Introduction

배경

기존 생성 모델에는 여러 방식이 있었음.

- **GAN**: 고품질 이미지를 만들 수 있지만 학습이 불안정할 수 있음
- **Autoregressive Model**: likelihood를 직접 계산할 수 있지만 순차적으로 생성해서 느림
- **Flow**: 정확한 likelihood 계산이 가능하지만 모델 구조에 제약이 있음
- **VAE**: 안정적으로 학습할 수 있지만 당시에는 생성 이미지 품질이 상대적으로 낮았음
- **Energy-based Model / Score Matching**: GAN과 비슷한 수준의 이미지 생성 성능을 보이기 시작함

이 논문에서는 **Diffusion Probabilistic Model**을 이용해 고품질 이미지 생성이 가능함을 보여줌.

Diffusion Model?

실제 이미지에 조금씩 노이즈를 추가해서 완전한 노이즈로 만든 다음, 그 과정을 거꾸로 되돌리는 방법을 모델이 학습하도록 하는 것
전체 과정은 두 부분으로 구성됨.

1) Forward Process

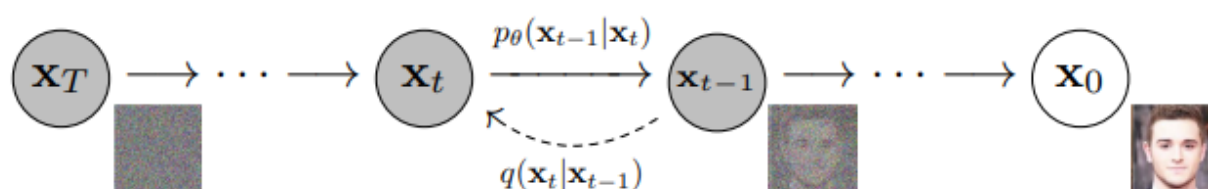
실제 이미지에 Gaussian noise를 조금씩 추가함

$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T$

- x_0 : 실제 이미지
- $x_1, x_2, \dots$: 점점 노이즈가 많아진 이미지
- x_T : 거의 완전한 Gaussian noise

Forward Process는 **학습하지 않고** 미리 정한 규칙에 따라 노이즈를 추가하는 과정

2) Reverse Process



반대로 노이즈에서 이미지를 복원함.

$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_1 \rightarrow x_0$

문제는 $p(x_{t-1}|x_t)$ 를 정확히 알 수 없다는 것임.

따라서 Neural Network를 이용해 $p_\theta(x_{t-1}|x_t)$ 를 학습함.

⇒ **Diffusion Model**의 실제 학습 대상은 노이즈를 어떻게 제거할 것인가임.

2. Background

Diffusion Model의 확률 구조

latent variable model: 관측 데이터는 x_0 이고, $x_1, x_2, \dots, x_T$ 는 latent variable처럼 취급됨.

모델의 전체 생성 분포: $p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t)$

- 마지막 noise는 $p(x_T) = \mathcal{N}(0, I)$ 에서 시작함.
- Reverse Process: $p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$
= Neural Network가 Reverse Process의 평균과 분산을 표현함.

Forward Process

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

- β_t : 각 timestep에서 얼마나 많은 noise를 추가할지 결정하는 값
- $\alpha_t = 1 - \beta_t$
- $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$

중요한 성질

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

- Forward Process를 실제로 $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots$ 순서대로 계산할 필요 없고 아무 timestep t 에 대해서도 x_0 에서 바로 x_t 를 만들 수 있음.
- 따라서 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \epsilon \sim \mathcal{N}(0, I)$ 로 만들 수 있음.

xt: 원본 이미지 x_0 와 Gaussian noise ϵ 를 적절한 비율로 섞은 결과

- 초기 timestep에서는 $\sqrt{\bar{\alpha}_t}$ 가 커서 원본 이미지 비중이 높음
- 반대로 t 가 커질수록 noise 비중이 증가

Variational Bound

Diffusion Model은 데이터의 negative log-likelihood에 대한 Variational Bound를 최소화하면서 학습함.

$$\mathbb{E}[-\log p_\theta(x_0)] \leq L$$

- 기본 목적함수: $L = \mathbb{E}_q \left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right]$
- $L = L_T + \sum_{t>1} L_{t-1} + L_0$
 - $L_T = D_{KL}(q(x_T|x_0) \parallel p(x_T))$: Forward Process의 마지막 x_T 가 $\mathcal{N}(0, I)$ 에 얼마나 가까운지 나타냄
 - $L_{t-1} = D_{KL}(q(x_{t-1}|x_t, x_0) \parallel p_\theta(x_{t-1}|x_t))$: 실제 Forward Process에서 얻을 수 있는 올바른 복원 분포와 모델이 예측한 Reverse Process가 얼마나 다른지 측정
 - L_0 : 마지막 단계에서 x_1 로부터 실제 데이터 x_0 를 얼마나 잘 복원하는지

3. Diffusion Models and Denoising Autoencoders

3.1 Forward Process and L_T

논문에서는 Forward Process의 variance β_t 를 학습하지 않고 **미리 정해진 상수로 고정함**.

따라서 Forward Process $q(x_t|x_{t-1})$ 에는 학습 가능한 parameter가 없음.

그러면 $L_T = D_{KL}(q(x_T|x_0) \parallel p(x_T))$ 역시 모델 parameter θ 와 관계없는 상수가 됨.

⇒ 학습 과정에서 L_T 는 무시할 수 있음.

3.2 Reverse Process and $L_{1:T-1}$

Reverse Process는 다음과 같이 Gaussian으로 정의:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

- 먼저 variance를 $\Sigma_{\theta}(x_t, t) = \sigma_t^2 I$ 처럼 **학습하지 않는 값으로 고정함**.
- 실험에서는 $\sigma_t^2 = \beta_t$ or $\sigma_t^2 = \tilde{\beta}_t$ 를 사용했으며 두 방식이 비슷한 결과를 보임.

즉 모델의 목표는 단순히 노이즈를 없애자가 아니라, 정확히는
현재 x_t 가 주어졌을 때 이전 상태 x_{t-1} 이 어떤 Gaussian 분포에서 나오는지를 학습하는 것
 Gaussian 분포를 정하려면 기본적으로 두 가지가 필요함.

- 평균 μ
- 분산 Σ

그런데 이 논문에서는 variance는 학습하지 않고 $\Sigma_{\theta}(x_t, t) = \sigma_t^2 I$ 처럼 **미리 고정해버림**.
 그러면 Reverse Process에서 모델이 실제로 배워야 할 것은 하나만 남음. $\rightarrow \mu_{\theta}(x_t, t)$
 즉 **이전 단계 x_{t-1} 가 중심으로 어디쯤 있어야 하는지 나타내는 평균임**.
 아래는 바로 이 mean을 어떻게 표현할지를 정하는 부분

현재 noisy image가 x_t 이고, 한 단계 전의 조금 덜 noisy한 image를 x_{t-1} 라고 하면 모델은

| x_t 가 주어졌을 때 x_{t-1} 는 대충 어디에 있어야 하지? 를 예측해야 함.

그 **대충 어디에 있어야 하지?** 를 나타내는 값이 바로 $\mu_{\theta}(x_t, t)$

그래서 처음부터 흐름을 다시 쓰면:

Reverse Process에서 무엇을 학습해야 하는가?

Reverse Process: $p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$

1. Gaussian 분포를 결정하려면 mean과 variance가 필요함.
 - $\rightarrow$ 논문에서는 variance $\Sigma_{\theta}(x_t, t)$ 는 학습하지 않고 고정함.
 - $\rightarrow$ 따라서 Neural Network가 실제로 학습해야 하는 핵심은 $\mu_{\theta}(x_t, t)$ (= **Reverse Process의 mean**)
2. 이 mean을 Neural Network가 직접 예측하게 할 수도 있지만, 저자들은 mean을 **noise ϵ** 을 이용해 표현하는 방식으로 바꿈.

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \epsilon_{\theta}(x_t, t) \right)$$
 = 실제 모델은 mean을 직접 출력하는 대신 $\epsilon_{\theta}(x_t, t)$ 를 출력함.
3. 그리고 그 예측한 noise를 위 식에 넣어서 mean을 계산

Forward Posterior

Forward Process에서는 x_0 과 x_t 를 알고 있으면 $q(x_{t-1}|x_t, x_0)$ 를 정확한 Gaussian 형태로 계산할 수 있음.

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$$

- $\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\tilde{\alpha}_{t-1}}\beta_t}{1-\tilde{\alpha}_t}x_0 + \frac{\sqrt{\tilde{\alpha}_t}(1-\tilde{\alpha}_{t-1})}{1-\tilde{\alpha}_t}x_t$
- $\tilde{\beta}_t = \frac{1-\tilde{\alpha}_{t-1}}{1-\tilde{\alpha}_t}\beta_t$

$\Rightarrow$ Reverse Process가 예측해야 하는 정답에 해당하는 Gaussian 분포를 계산할 수 있음.

1. Mean 직접 예측

Reverse Process의 loss : $L_{t-1} = \mathbb{E} \left[\frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(x_t, x_0) - \mu_{\theta}(x_t, t)\|^2 \right] + C$

따라서 그냥 $\mu_{\theta}(x_t, t)$ 가 $\tilde{\mu}_t(x_t, x_0)$ 를 직접 맞히도록 학습하면 됨.

하지만 논문에서는 더 효과적인 parameterization을 제안함.

2. Noise ϵ 예측

$x_t = \sqrt{\alpha_t}x_0 + \sqrt{1-\alpha_t}\epsilon$ 를 x_0 에 대해 정리하면

$$\rightarrow x_0 = \frac{1}{\sqrt{\alpha_t}} (x_t - \sqrt{1-\alpha_t}\epsilon)$$

이 값을 Forward Posterior Mean 식에 넣으면 Reverse Mean을 noise ϵ 로 표현할 수 있음.

결과적으로 $\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \epsilon_{\theta}(x_t, t) \right)$ 로 parameterization함.

- 여기서 $\epsilon_\theta(x_t, t)$ 는 Neural Network가 예측한 noise

즉 모델은 현재 **noisy image x_t** 에 어떤 **Gaussian noise**가 들어갔는지 예측하도록 학습됨.

왜 굳이 Noise를 예측할까?

원래 목표는 $x_t \rightarrow x_{t-1}$ 로 되돌리는 건데 x_{t-1} 자체를 직접 예측하는 대신 x_t 에 들어간 noise를 찾아내고 그 noise를 제거하는 문제로 바꾼 것

이렇게 하면 학습식이 매우 간단한 MSE 형태가 됨.

Noise Prediction Loss

위 parameterization을 기존 Variational Bound에 넣으면 $\mathbb{E} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \|\epsilon - \epsilon_\theta(x_t, t)\|^2 \right]$ 형태가 됨.

= 실제로 넣었던 noise와 모델이 예측한 noise의 차이를 줄이는 문제

Denoising Score Matching과의 연결

Diffusion Model ↔ **Denoising Score Matching** ↔ **Langevin Dynamics**

이 noise prediction objective는 여러 noise level에 대해 수행되는 **Denoising Score Matching**과 유사함.

논문에서는 이를 통해 Diffusion Model과 Score-based Generative Model 사이의 직접적인 연결을 보여줌.

또한 Reverse Sampling 과정은 **Langevin Dynamics**와 유사한 형태를 가짐.

3.3 Data Scaling, Reverse Process Decoder, and L_0

이미지의 pixel 값은 원래 $\{0, 1, \dots, 255\}$ 범위를 가지는데 논문에서는 이를 $[-1, 1]$ 범위로 scaling함.

이유는 Reverse Process Neural Network가 항상 비슷한 scale의 입력을 받도록 하기 위함

- Forward Process가 충분히 진행되면 $x_T \approx \mathcal{N}(0, I)$ 가 되는데, 이미지 데이터 역시 $[-1, 1]$ 범위로 맞춰두면 전체 과정의 scale을 일관되게 유지할 수 있음.

$p_\theta(x_0|x_1)$ 은 Gaussian에서 유도된 discrete decoder로 정의

3.4 Simplified Training Objective

원래 Variational Bound를 사용하면 timestep마다 서로 다른 weight가 붙음.

하지만 저자들은 실제 실험에서 이를 단순화한 objective가 생성 품질에 더 좋다는 것을 발견함.

최종적으로 다음 Loss를 사용 : $L_{\text{simple}}(\theta) = \mathbb{E}_{t, x_0, \epsilon} \left[\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2 \right]$

식 쉽게 풀기

- 먼저 실제 이미지 x_0 를 하나 가져옴 + timestep t 를 무작위로 하나 선택함
- Gaussian noise ϵ 생성 $\rightarrow \epsilon \sim \mathcal{N}(0, I)$
- 그 noise를 이미지에 섞어서 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$ 생성
- 이 noisy image와 timestep t 를 모델에 넣음 $\rightarrow \epsilon_\theta(x_t, t)$
- 모델은 이 이미지에 들어간 noise가 뭔지 예측함.
- 그 후 실제 noise와 비교하고 $\rightarrow \|\epsilon - \epsilon_\theta(x_t, t)\|^2 \rightarrow$ 이 차이가 작아지도록 학습함.

왜 Simplified Objective가 더 좋았나?

원래 Variational Bound에는 timestep마다 다른 weight가 붙음.

논문에서는 이를 제거함으로써 작은 t 의 아주 쉬운 denoising 문제에 지나치게 집중하지 않고, noise가 많이 섞인 더 어려운 denoising 문제에도 충분히 학습하도록 함.

결과적으로 likelihood는 조금 나빠졌지만 **sample quality**는 크게 향상됨.

1) DDPM Training

Algorithm 1 Training

```
1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
        $\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$ 
6: until converged
```

1. 실제 이미지 선택 $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
2. timestep 무작위 선택 $t \sim \text{Uniform}(\{1, \dots, T\})$
3. Gaussian noise 생성 $\epsilon \sim \mathcal{N}(0, \mathbf{I})$
4. noisy image 생성 $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
5. 모델이 noise 예측 $\epsilon_{\theta}(x_t, t)$
6. MSE 계산 $\left\| \epsilon - \epsilon_{\theta}(x_t, t) \right\|^2$
7. Gradient Descent 위 Loss가 작아지도록 θ 업데이트

중요한 점

Forward Process가 $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T$ 형태라고 해서 학습할 때 매번 1000단계를 전부 거치는 것이 아니라 임의의 t 하나를 선택한 뒤 $x_0 \rightarrow x_t$ 를 직접 계산함.

⇒ 학습 시에는 한 번의 식으로 원하는 timestep의 noisy image를 만들 수 있음.

2) Sampling

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

1. 학습이 끝난 후 실제 이미지를 생성할 때는 완전한 Gaussian noise에서 시작함 $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$
2. $\mathbf{x}_T \rightarrow \mathbf{x}_{T-1} \rightarrow \dots \rightarrow \mathbf{x}_0$ 순서로 하나씩 denoising함

한 단계는 다음과 같음:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, \mathbf{I}) \quad (\text{단, } t=1 \text{ 일 때는 } z=0)$$

의미

1. 먼저 $\epsilon_{\theta}(x_t, t)$ 를 통해 현재 이미지에 존재하는 noise를 예측함.
2. 그 noise를 제거해서 이전 timestep 방향으로 이동함.
3. 그리고 적절한 크기의 새로운 random noise $\sigma_t z$ 를 추가함.
4. 이를 계속 반복하면 $x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0$ 가 됨.

3) Training vs Sampling

Training	Sampling
실제 이미지 x_0 이 있음	실제 이미지 없음
임의의 t 하나만 선택	$T \rightarrow 1$ 전체 timestep 진행
실제 noise ϵ 를 알고 있음	실제 noise를 모름
noise를 맞히도록 학습	예측한 noise를 제거
$x_0 \rightarrow x_t$ 를 한 번에 계산 가능	매 timestep을 순차적으로 진행해야 함

4. Experiments

기본 설정

- 모든 실험에서 $T=1000$ 을 사용함.
- Forward Process variance는 $\beta_1 = 10^{-4}$ 에서 $\beta_T=0.02$ 까지 선형적으로 증가
= 처음에는 아주 작은 양의 noise를 추가하고 timestep이 커질수록 더 많은 noise를 추가

모델 구조

Reverse Process의 Neural Network는 **U-Net 기반 구조**를 사용

주요 구성:

- U-Net
- Residual Block
- Group Normalization
- Self-Attention
- Sinusoidal Position Embedding

특히 timestep t 도 Neural Network에 알려줘야 하는데, 같은 noisy image라도 현재 diffusion 과정의 몇 번째 단계인가에 따라 제거해야 할 noise의 수준이 다르기 때문

그래서 Transformer의 sinusoidal position embedding을 이용해 t 정보를 모델에 전달함.

4.1 Sample Quality

원래 Variational Bound L 로 학습하면 log-likelihood는 더 좋아지지만

L_{simple} 을 사용하면 이미지의 **생성 품질은 더 좋아짐**.

⇒ likelihood를 더 잘 최적화한다고 해서 반드시 사람이 보기에 더 좋은 이미지가 생성되는 것은 아님.

4.2 Reverse Process Parameterization and Training Objective Ablation

논문에서는 Reverse Process에서 무엇을 예측하는 것이 좋은지 비교함.

단순히 noise를 예측하는 것만으로 성능이 좋아진 것이 아니라 **noise prediction**과 **simplified objective**의 조합이 특히 효과적이었음

+ Variance를 직접 학습하면?

Reverse Process의 variance $\Sigma_\theta(x_t, t)$ 도 Neural Network가 학습하도록 실험했는데

결과적으로

- 학습이 불안정해지고
- sample quality가 떨어짐

따라서 이 논문에서는 variance를 고정해서 사용함.

4.3 Progressive Coding

Diffusion Model의 흥미로운 특징 중 하나는 이미지가 점진적으로 만들어진다는 점임.

Sampling 과정은 대략 Noise → 큰구조 → 세부구조 → 완성이미지 형태로 진행



처음에는 이미지가 거의 noise처럼 보이다가 Reverse Step이 진행되면서 전체적인 형태가 먼저 나타나고, 마지막에 세부적인 texture와 형태가 추가됨.

Progressive Generation

- Reverse Process 초반: 전체 형태, 물체 위치, 큰 색상 구조 등이 형성
- Reverse Process 후반: texture, 세부 윤곽, 작은 특징 등이 추가됨.

⇒ **coarse-to-fine** 방식으로 이미지를 생성하는 경향을 보임.

Progressive Compression

저자들은 이를 compression 관점에서도 해석함.

Diffusion의 latent variable x_t 는 timestep에 따라 서로 다른 수준의 정보를 가지고 있음.

- 큰 t: 세부 정보는 거의 사라짐, 큰 구조만 남음
- 작은 t: 세부 정보까지 많이 포함됨

따라서 Reverse Process가 진행되면서 이미지 정보가 점진적으로 복원되는 구조를 **progressive decompression**처럼 볼 수 있음.

Autoregressive Model과의 관계

논문에서는 Diffusion Model을 특수한 형태의 autoregressive model로도 해석함.

- 일반적인 Autoregressive Model: $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow \dots$ 처럼 픽셀이나 token을 정해진 순서대로 하나씩 생성
- Diffusion Model은 특정 픽셀 순서가 아니라 전체 이미지에 존재하는 정보를 noise level에 따라 점진적으로 생성

⇒ 저자들은 이를 **generalized bit ordering**을 가진 **autoregressive decoding**처럼 해석

즉 Autoregressive Model이 위치 순서에 따라 정보를 생성한다면, Diffusion은 noise level에 따라 정보를 점진적으로 복원한다고 볼 수 있음.

4.4 Interpolation

Diffusion Model의 latent space에서도 interpolation이 가능함.

1. 두 이미지 x_0 , x'_0 를 Forward Process를 이용해 x_t , x'_t 로 변환함.
2. 그다음 latent 사이를 선형 interpolation함 : $\bar{x}_t = (1 - \lambda)x_t + \lambda x'_t$
3. 그리고 이 latent를 Reverse Process를 통해 다시 이미지로 복원함.

논문의 CelebA-HQ 실험에서는 자세, 피부톤, 헤어스타일, 표정, 배경 등이 자연스럽게 변화하는 것을 확인함.

timestep에 따른 차이

- 작은 t: 원본 정보가 많이 남아 있음
→ 두 이미지 사이의 세부적인 변화
- 큰 t: 원본 정보가 더 많이 제거됨
→ 더 큰 의미 수준의 변화

특히 t=1000에서는 원본 정보가 거의 사라져 두 이미지를 단순히 섞는 것이 아니라 새로운 sample에 가까운 결과를 만들 수 있음.

5. Related Work

Diffusion Model은 여러 생성 모델과 연결됨.

VAE와의 관계

Diffusion 역시 latent variable model이라는 점에서 VAE와 비슷함.

but

1. VAE: Encoder $q_\phi(z|x)$ 를 학습
vs Diffusion: Forward Process $q(x_{1:T}|x_0)$ 가 미리 정해져 있음
2. 학습 가능한 encoder가 없음

즉 Diffusion은 latent variable model이지만 inference process 자체를 학습하지 않는다는 특징이 있음.

Flow와의 관계

Flow 역시 여러 단계의 변환을 통해 $x \leftrightarrow z$ 를 연결함.

- 하지만 Flow는 보통 invertible transformation을 사용해야 하는 구조적 제약이 있음.
- Diffusion은 stochastic process를 사용하므로 이러한 형태의 가역성 제약이 필요하지 않음.

Score Matching과의 관계

Noise prediction parameterization을 사용하면 DDPM의 목적함수가 Denoising Score Matching과 연결됨.

$\epsilon_\theta(x_t, t)$ 를 잘 예측하는 것은 각 noise level에서 데이터 분포의 score를 학습하는 것과 관련됨.

Langevin Dynamics와의 관계

DDPM의 Reverse Sampling 과정은 Langevin Dynamics와 유사함.

하지만 기존 Score-based Model과 달리 DDPM에서는

- noise scale
- sampling coefficient
- step별 variance

등이 Forward Process의 β_t 로부터 체계적으로 유도됨.

즉 Sampling Algorithm이 학습 이후 따로 붙는 것이 아니라 처음부터 모델의 확률 구조에 포함되어 있음.

NCSN과의 차이

NCSN 역시 여러 noise level에서 score를 학습하지만..

NCSN

1. Score Network를 학습함
2. 학습 이후 Langevin Sampling Algorithm을 별도로 설정함

DDPM

1. Forward Process 정의
2. Reverse Process 정의
3. Variational Inference로 Reverse Process 자체를 학습

⇒ DDPM은 **Sampling Process 자체가 하나의 latent variable model 안에 포함되어 학습됨.**

6. Conclusion

이 논문의 핵심은 Diffusion Model이 단순한 이론적 생성 모델이 아니라 **고품질 이미지 생성 모델로 실제 사용 가능함**을 보여준 것임.

특히 다음 관계를 제시함.

Diffusion Model

↔ Variational Inference

- ↔ Denoising Score Matching
- ↔ Langevin Dynamics
- ↔ Autoregressive Model
- ↔ Progressive Compression